OOP – Übungsserie Kapitel 1–4

Drei Übungsblätter im Stil von OOP01–OOP04 · C++17 · pbma · Google Test · valgrind · Stand 03.11.2026

Inhalt

Übersicht und allgemeine Hinweise · **Blatt 1** Kapitel 1–2 (4 Aufgaben) · **Blatt 2** Kapitel 3 (4 Aufgaben) · **Blatt 3** Kapitel 4 (4 Aufgaben)

Tests, Daten und Vorlagen liegen in den Ordnern `blatt1/` bis `blatt3/` und `vorlagen/`. Lösungen und Lösungshinweise liegen getrennt in `loesungen/`.

Übersicht

Drei Blätter im Stil von OOP01–OOP04. Jedes Blatt deckt einen Stoffblock ab und braucht etwa 90 bis 120 Minuten. Die Aufgaben auf Blatt 2 und 3 bauen teilweise aufeinander auf, wie auf den echten Blättern.

Blatt	Stoff	Aufgaben	Ordner
1	Kapitel 1–2: C-Wiederholung, Namensräume, Referenzen, const, Überladung, Default-Werte, Freispeicher, Ein-/Ausgabe	1 ggT-Bibliothek und Programm `paare` · 2 Werkzeugfunktionen · 3 Übersetzt es? · 4 Pascalsches Dreieck	`blatt1/`
2	Kapitel 3: Klassen, Konstruktoren, Initialisierungsliste, Delegation, const, static, friend, explicit	1 Intervall · 2 Belegungsplan und Programm `raum` · 3 Sensor mit static · 4 So kompakt wie möglich	`blatt2/`
3	Kapitel 4 (mit 2.9): Freispeicher in Klassen, Kopierkonstruktor, Zuweisung, Copy & Swap, Dreierregel, Ressourcen	1 Bitmenge · 2 Zeilenpuffer und Programm `rueckwaerts` · 3 CSV-Leser · 4 Speicherbild	`blatt3/`

Allgemeine Hinweise (gelten für alle Blätter)

- Für jede Aufgabe legen Sie ein eigenes Eclipse-Projekt an. Programme mit `main` und Unit-Tests gehören in getrennte Projekte. Kopieren Sie Ihre Implementierungsdateien in beide Projekte.
- `pbma.h/.cpp`, `alltest.h` und `gtest_main.cpp` liegen in `vorlagen/`. Für die Tests stellen Sie den Linker auf `-pthread` und die Library `gtest` (wie auf Blatt 2).
- Übersetzen Sie mit `-std=c++17 -Wall -Wextra -pedantic`. Ihr Code soll ohne Warnung übersetzen.
- Verwenden Sie `const` wann immer sinnvoll möglich und die Elementinitialisierungsliste, wo immer es geht. Fehlertexte übernehmen Sie zeichengenau.
- Prüfen Sie jedes Programm und jedes Testprojekt mit valgrind. Ohne Fehler und ohne Speicherleck ist die Aufgabe erst fertig.
- Wo keine Prototypen angegeben sind, leiten Sie sie aus dem Text und aus dem Test ab.

Blatt 1 – Kapitel 1 und 2

Aufgabe 1. *(ca. 30 min · Ordner `blatt1/a1_paare`)* Erstellen Sie eine Bibliothek `zahlen` (`zahlen.h`, `zahlen.cpp`), deren Funktionen alle im Namensraum `zt` liegen.

- `ggt_rek` berechnet den größten gemeinsamen Teiler zweier `int` rekursiv mit dem Modulo-Verfahren aus Blatt 1, `ggt_it` dasselbe iterativ. Beide arbeiten auch mit negativen Zahlen, das Ergebnis ist nie negativ. Der ggT von `a` und 0 ist `|a|`.
- `kgv` berechnet das kleinste gemeinsame Vielfache. Es ist nie negativ und ist 0, wenn eine der Zahlen 0 ist. Wählen Sie den Rückgabetyp so, dass das kgV von 1000000 und 999999 noch passt.
- `teilerfremd` liefert, ob der ggT genau 1 ist.

Testen Sie die Bibliothek mit `test_zahlen.cpp`.

Schreiben Sie dann ein Programm `paare` (eigenes Projekt). Es liest mit `read_ints` aus `pbma` eine Datei mit ganzen Zahlen, deren Name das erste Positionsargument ist (Vorgabe `paare.dat`). Je zwei aufeinanderfolgende Zahlen bilden ein Paar. Ist die Anzahl ungerade oder 0, bricht das Programm mit einer `std::runtime_error` ab, die in `main` gefangen und auf `std::cerr` ausgegeben wird. Für jedes Paar berechnen Sie ggT und kgV. Stellen Sie dabei sicher, dass `ggt_rek` und `ggt_it` dasselbe Ergebnis liefern. Mit dem Flag `-v` wird jedes Paar ausgegeben, beide Zahlen jeweils in Breite 8. Danach kommen die Anzahl der Paare, die Anzahl der teilerfremden Paare und die Summe aller kgV. Mit dem Flag `-zeit` vergleichen Sie zusätzlich die Laufzeiten von `ggt_rek` und `ggt_it` mit `Timer` auf 1 000 000 Paaren aus `create_same_randints(2000000, 1, 1000000)`. Gemessen wird nur die Berechnung.

```
$ ./paare -v
      45      10: ggT 5, kgV 90
      12      18: ggT 6, kgV 36
      17       5: ggT 1, kgV 85
      -8      12: ggT 4, kgV 24
       0       7: ggT 7, kgV 0
 1000000  999999: ggT 1, kgV 999999000000
      21      14: ggT 7, kgV 42
      35      64: ggT 1, kgV 2240
Paare: 8
Teilerfremd: 3
Summe kgV: 999999002517
$ ./paare ungerade.dat
Fehler: ungerade Anzahl von Zahlen in ungerade.dat
```

Debuggen Sie `ggt_rek(45, 10)` und notieren Sie für jede Rekursionsstufe die Werte von `a` und `b`. Wie viele Aufrufe von `ggt_rek` stehen auf dem Call-Stack, wenn `b == 0` erreicht ist?

Aufgabe 2. *(ca. 25 min · `blatt1/a2_werkzeug`)* Schreiben Sie eine Bibliothek `werkzeug` im Namensraum `wz` und testen Sie sie mit `test_werkzeug.cpp`.

- `sortiere3` sortiert drei `int`-Variablen des Aufrufers aufsteigend.
- `element` liefert eine Referenz auf ein Element eines `int`-Feldes mit gegebener Länge (`size_t`). Der Index (`long`) darf negativ sein und zählt dann vom Ende: -1 ist das letzte Element. Bei ungültigem Index wird `std::runtime_error` mit dem Text `index out of range` geworfen.
- `teile` berechnet ganzzahligen Quotienten und Rest und liefert beide über Parameter zurück. Bei Divisor 0 liefert die Funktion `false`, und die beiden Ergebnisparameter bleiben unverändert. Sonst liefert sie `true`.
- `zaehle` zählt, wie oft ein Zeichen in einem Text vorkommt. Ohne Angabe des Zeichens wird das Leerzeichen gezählt. Der Text darf dabei nicht kopiert werden.
- `wiederhole` liefert einen String `n`-mal hintereinander, getrennt durch einen Trenner. Ohne Angabe ist `n` gleich 2 und der Trenner leer. Ist `n` negativ, wird `negative count` geworfen.

Warum muss `zaehle` den Text als `const std::string&` erhalten und nicht als `std::string&`? Probieren Sie den Aufruf `wz::zaehle("Ene Mene Muh")` mit beiden Varianten aus.

Aufgabe 3. *(ca. 20 min, auf Papier · `blatt1/a3_orakel/orakel.cpp`)* Gegeben ist das Programm in `orakel.cpp` mit den Zeilen A bis P in `main` (Auszug):

```
namespace a { int wert = 1; int f(int x) { return x + 10; } }
namespace b { int wert = 2; int f(double x) { return static_cast<int>(x) + 20; } }
int wert = 3;
void p(int* q) { std::cout << "int* "; *q += 1; }
void p(const int* q) { std::cout << "const int* " << *q << " "; }
int m(int x, int y = 4, int z = 5) { return x * 100 + y * 10 + z; }
int& groesser(int& x, int& y) { return x > y ? x : y; }
using namespace std;

int i = 7, j = 9;  const int k = 8;  int* const pc = &i;  const int* cp = &i;
A  cout << a::wert + b::wert + ::wert           B  cout << wert
C  { using namespace a; cout << f(1); }
D  { using namespace a; using namespace b; cout << f(1.5) << " " << f(1); }
E  { using namespace a; using namespace b; cout << wert; }
F  *pc = 11;     G  pc = &j;     H  *cp = 12;     I  cp = &k;
J  p(&i); p(&k); p(cp); cout << i
K  cout << m(1) << " " << m(1, 2) << " " << m(1, 2, 3)      L  cout << m()
M  groesser(i, j) = 0; cout << i << " " << j               N  groesser(i, 5) = 1;
O  string s = "OOP"; string& r = s; r += "!"; cout << s     P  int& kein;
```

Übersetzt es korrekt? Wenn nicht, welche Fehler finden Sie? Gehen Sie davon aus, dass nach jedem Fehler die entsprechende Zeile einfach fehlt und es weitergehen würde. Was ist dann die Ausgabe? Prüfen Sie Ihre Antworten mit dem Compiler: fehlerhafte Zeile auskommentieren, neu bauen.

Aufgabe 4. *(ca. 25 min · `blatt1/a4_dreieck`)* Schreiben Sie ein Programm `dreieck`, das die ersten `n` Zeilen des Pascalschen Dreiecks berechnet und ausgibt. `n` ist das erste Positionsargument (Vorgabe 5, erlaubt 1 bis 20, sonst Abbruch mit Meldung). Speichern Sie das Dreieck in einem Feld von Zeilen im Freispeicher: ein `int**` mit `n` Zeigern, wobei Zeile `i` genau `i + 1` Elemente hat. Reservieren Sie also nicht mehr Speicher als nötig. Jedes Element ist die Summe der beiden darüberliegenden, die Ränder sind 1. Lagern Sie Erzeugen, Ausgeben und Freigeben in eigene Funktionen aus. Die Ausgabe erfolgt zeilenweise, jede Zahl in Breite 6. Mit dem Flag `-summe` folgt am Zeilenende `=` und die Zeilensumme. Zum Schluss geben Sie aus, wie viele Bytes Sie im Freispeicher angefordert haben (Zeiger und `int`-Werte, mit `sizeof` berechnet).

```
$ ./dreieck 6 -summe
     1  = 1
     1     1  = 2
     1     2     1  = 4
     1     3     3     1  = 8
     1     4     6     4     1  = 16
     1     5    10    10     5     1  = 32
Bytes: 132
```

Was gibt das Programm für `n = 5` als Bytezahl aus (64 Bit)? Rechnen Sie es vorher von Hand aus. Lassen Sie das Programm mit valgrind laufen. Was meldet valgrind, wenn Sie in der Freigabe das `delete[] zeilen;` am Ende weglassen, und was, wenn Sie stattdessen die Schleife weglassen?

Blatt 2 – Kapitel 3

Aufgabe 1. *(ca. 30 min · `blatt2/a1_intervall`)* Erstellen Sie eine Klasse `Intervall` (`Intervall.h`, `Intervall.cpp`), die ein geschlossenes Intervall ganzer Zahlen `[von, bis]` kapselt. Beide Grenzen gehören dazu, und es gilt immer `von <= bis`. Die Prototypen der Methoden sind nicht angegeben, da Sie die korrekten Prototypen aus dem Text herauslesen sollen.

Man kann beim Erstellen beide Grenzen angeben. Werden sie vertauscht angegeben, werden sie getauscht. Wird nur eine Grenze angegeben, entsteht ein Intervall aus genau dieser Zahl. Ohne Angabe ist es `[0, 0]`. Realisieren Sie das mit Delegation (Skript 3.2) statt doppeltem Code. Eine einzelne Zahl darf überall dort implizit als Intervall verwendet werden, wo ein Intervall erwartet wird.

Man kann die Grenzen mit `von` und `bis` abfragen und mit `laenge` die Anzahl der enthaltenen Zahlen. `enthaelt` prüft, ob eine Zahl oder ein ganzes anderes Intervall im Intervall liegt (zwei Methoden gleichen Namens). `ueberlappt` prüft, ob zwei Intervalle mindestens eine Zahl gemeinsam haben. `verschiebe` verschiebt beide Grenzen um einen (auch negativen) Betrag und ist verkettbar: `i.verschiebe(2).verschiebe(-10)`. `vereinigung` liefert ein neues Intervall, das beide umfasst, und ändert keines der beiden. Das geht nur, wenn sie sich überlappen oder direkt aneinanderstoßen (`[3, 7]` und `[8, 9]`). Sonst wird `std::runtime_error("disjoint")` geworfen. `out` gibt das Intervall auf einen Strom aus (Vorgabe `std::cout`), im Format `[3, 7]`.

Testen Sie mit `test_intervall.cpp`. Warum würde der Aufruf `c.enthaelt(5)` mehrdeutig, wenn Sie statt zweier `enthaelt`-Methoden nur `enthaelt(const Intervall&)` hätten? Wird er überhaupt mehrdeutig? Probieren Sie es aus.

Aufgabe 2. *(ca. 35 min · `blatt2/a2_raum`)* Ein Seminarraum wird minutengenau gebucht. Die Zeiten sind Minuten ab 0:00 Uhr, also 480 für 8:00. Erstellen Sie eine Klasse `Belegungsplan`, die Ihre Klasse `Intervall` verwendet. Kopieren Sie dazu `Intervall.h/.cpp` aus Aufgabe 1 in das Projekt.

Ein Belegungsplan hat einen Raumnamen und eine Öffnungszeit (Vorgabe 8:00 bis 19:59, also `[480, 1199]`). Beide sind nach der Erzeugung unveränderlich. Ein String soll nicht unbemerkt zu einem Belegungsplan werden. Die Reservierungen speichern Sie in einem `std::vector<Intervall>`.

`reservieren` nimmt ein Zeitintervall an, wenn es vollständig in der Öffnungszeit liegt und sich mit keiner bestehenden Reservierung überschneidet. Es liefert zurück, ob das geklappt hat. `stornieren` entfernt die Reservierung, die eine gegebene Minute enthält, und meldet, ob es eine gab. Man kann die Anzahl der Reservierungen abfragen, die `i`-te Reservierung ohne Kopie lesen (`index out of range` bei ungültigem Index), sowie belegte und freie Minuten der Öffnungszeit. Raumname und Öffnungszeit sind ebenfalls ohne Kopie abfragbar. Testen Sie mit `test_belegungsplan.cpp`.

Schreiben Sie dann ein Programm `raum`. Es liest mit `read_ints` Reservierungswünsche als Paare von Minuten aus einer Datei (erstes Positionsargument, Vorgabe `wuensche.dat`) und versucht sie der Reihe nach zu buchen. Der Raumname kommt aus der Option `-raum=Name` (Vorgabe `A101`). Pro Wunsch wird das Intervall als Uhrzeit `hh:mm-hh:mm` ausgegeben, dahinter `ok` oder `abgelehnt`. Schreiben Sie dafür eine Funktion, die Minuten in einen String `hh:mm` umwandelt (`ostringstream`, `setw`, `setfill`).

```
$ ./raum
08:00-09:29 ok
09:30-10:59 ok
10:00-10:29 abgelehnt
11:00-11:59 ok
07:50-08:20 abgelehnt
13:00-13:59 ok
12:30-13:10 abgelehnt
19:00-19:59 ok
19:00-20:00 abgelehnt
Raum A101: 5 Reservierungen, 360 min belegt, 360 min frei
```

Warum muss `Belegungsplan` seine Attribute in der Elementinitialisierungsliste setzen? Was passiert, wenn `Intervall` keinen Standardkonstruktor hätte?

Aufgabe 3. *(ca. 30 min · `blatt2/a3_sensor`)* Schreiben Sie eine Klasse `Sensor`. Jeder Sensor erhält bei der Erzeugung automatisch eine eindeutige, unveränderliche Kennung, beginnend bei 1. Er hat einen Messwert (`double`, optional bei der Erzeugung, sonst 0). Ein Messwert soll nicht unbemerkt in einen Sensor umgewandelt werden.

- Eine **Kopie** eines Sensors ist ein neuer Sensor. Sie erhält deshalb eine neue Kennung, übernimmt aber den Messwert.
- Bei einer **Zuweisung** behält der Sensor seine Kennung und übernimmt nur den Messwert.
- `messe` setzt den Messwert. `kennung` und `messwert` sind auch für konstante Sensoren abfragbar.
- Mit statischen Methoden kann man ohne Instanz abfragen, wie viele Sensoren aktuell existieren (`aktive`) und wie viele höchstens gleichzeitig existiert haben (`hoechststand`).
- Die freie Funktion `tausche_messwerte` vertauscht die Messwerte zweier Sensoren und greift dazu direkt auf die privaten Attribute zu.

Lagern Sie das Vergeben der Kennung und das Mitzählen in eine private statische Hilfsmethode aus, die beide Konstruktoren benutzen. Testen Sie mit `test_sensor.cpp`.

Verfolgen Sie danach in einem eigenen Projekt mit einem kleinen Hauptprogramm im Debugger oder durch Ausgaben, was beim Füllen eines `std::vector<Sensor>` passiert. Legen Sie in einem Block einen leeren `vector<Sensor>` an, rufen Sie dreimal `push_back(Sensor(i * 1.5))` auf und geben Sie nach jedem Aufruf `aktive()` und `hoechststand()` aus. Geben Sie danach die Kennungen der Sensoren im Vektor aus. Wiederholen Sie das Ganze mit `reserve(3)` vor der Schleife. Erklären Sie jede Zahl. Insbesondere: Warum stehen im Vektor nicht die Kennungen 1, 2, 3?

Aufgabe 4. *(ca. 20 min · `blatt2/a4_kompakt`)* Schreiben Sie Code so kompakt wie möglich, sodass die Programme funktionieren und die erwartete Ausgabe haben.

a) `farbe.cpp`: Eine `Farbe` hat die öffentlichen Anteile `r`, `g`, `b`. Die Zeichen `'r'`, `'g'`, `'b'` stehen für reines Rot, Grün oder Blau (Anteil 255), jedes andere Zeichen für Schwarz.

```
Farbe schwarz;  Farbe grau(128);  Farbe lila(128, 0, 128);  Farbe rot = 'r';
zeige(schwarz); zeige(grau); zeige(lila); zeige(rot);
zeige('b'); zeige(Farbe(1, 2)); zeige(64); zeige('x');
// 0,0,0 128,128,128 128,0,128 255,0,0 0,0,255 1,2,0 64,64,64 0,0,0
```

Was würde `zeige('r')` ausgeben, wenn es den Konstruktor für Zeichen nicht gäbe?

b) `zaehlwerk.cpp`: Verwenden Sie das Schlüsselwort `class`, so wenige Methoden wie möglich und die Elementinitialisierungsliste.

```
Zaehlwerk a, b(10);  const Zaehlwerk c(5);
a.tick().tick();  b.tick();
cout << a.stand() << " " << b.stand() << " " << c.stand() << " " << Zaehlwerk::ticks(); // 2
11 5 3
// c.tick();         muss einen Compilerfehler geben
// Zaehlwerk d = 7;  muss einen Compilerfehler geben
```

Blatt 3 – Kapitel 4

Aufgabe 1. *(ca. 35 min · `blatt3/a1_bitmenge`)* Entwickeln Sie eine Klasse `Bitmenge`, die eine feste Anzahl `n` von Bits (Typ `size_t`) speichert. Ablegen soll sie diese platzsparend in einem dynamisch angelegten Feld `unsigned char*` mit `(n + 7) / 8` Bytes. Bit `i` liegt im Byte `i / 8` an Bitposition `i % 8` (Bitoperatoren `|`, `&`, `~`, `<<`, `>>` wie in C). Die Klasse verfügt über:

- einen Konstruktor mit der Anzahl der Bits. Alle Bits sind anfangs 0. Bei 0 Bits wird `std::runtime_error("invalid size")` geworfen. Eine Zahl soll nicht unbemerkt zu einer Bitmenge werden.
- `groesse`, `setze(i)`, `loesche(i)`, `ist_gesetzt(i)` und `anzahl` (Anzahl gesetzter Bits). Bei ungültigem Index wird `std::runtime_error` mit der Fehlermeldung `index out of range` geworfen.
- `vereinige(other)`: setzt alle Bits, die in `other` gesetzt sind, ist verkettbar und wirft `size mismatch` bei unterschiedlicher Größe.
- `out`: gibt die Bits in der Reihenfolge 0 bis `n − 1` als Zeichen `0`/`1` auf einen Strom aus, zum Beispiel `0100000010`.
- Kopierkonstruktor, Zuweisungsoperator und Destruktor. Realisieren Sie die Zuweisung mit **Copy & Swap** (Skript 4.4).

Testen Sie mit `test_bitmenge.cpp`. Alle Tests prüfen mit `AllTest` auch den Speicher. Zeichnen Sie für `Bitmenge b(10); b.setze(1); b.setze(8);` die beiden Bytes als Bitmuster. Warum braucht Ihr Zuweisungsoperator keinen Test auf Selbstzuweisung?

Aufgabe 2. *(ca. 40 min · `blatt3/a2_zeilen`)* Schreiben Sie eine Klasse `Zeilenpuffer`, die Textzeilen in einem selbst verwalteten Feld `std::string*` im Freispeicher hält (kein `vector`). Beim Erzeugen kann man die Anfangskapazität angeben (Vorgabe 2, 0 ist ungültig: `invalid capacity`).

- `anhaengen` hängt eine Zeile an. Ist das Feld voll, wird die Kapazität verdoppelt.
- `size` und `kapazitaet` geben Auskunft über den Füllstand.
- `zeile(i)` erlaubt bei veränderbaren Puffern das Ändern der Zeile, bei konstanten nur das Lesen. Kopiert wird dabei nichts. Bei ungültigem Index wird `index out of range` geworfen.
- `entferne(i)` löscht Zeile `i` und schiebt alle folgenden nach vorn.
- `finde(wort, ab)` liefert den Index der ersten Zeile ab Index `ab` (Vorgabe 0), die `wort` enthält, oder `size()`, wenn es keine gibt.
- Kopierkonstruktor, Zuweisungsoperator und Destruktor. Realisieren Sie die Zuweisung diesmal **klassisch**: Identitätstest, Freigeben, Neuanlegen, Kopieren.

Testen Sie mit `test_zeilenpuffer.cpp`. Schreiben Sie dann ein Programm `rueckwaerts`. Es liest eine Textdatei (erstes Positionsargument, Vorgabe `gedicht.txt`) zeilenweise mit `getline` in einen `Zeilenpuffer` ein und gibt die Zeilen in umgekehrter Reihenfolge aus. Vor jeder Zeile steht ihre ursprüngliche Nummer in Breite 3 und `:` (vgl. Skript 2.9.5). Leere Zeilen werden nur mit dem Flag `-leer` ausgegeben. Mit der Option `-suche=Wort` werden nur Zeilen ausgegeben, die das Wort enthalten. Am Ende steht, wie viele Zeilen ausgegeben wurden. Achten Sie bei der Rückwärtsschleife mit `size_t` auf den Unterlauf.

```
$ ./rueckwaerts                                  $ ./rueckwaerts -suche=Fehler
  8: und ploetzlich ergibt alles Sinn.             4: Ich lese nur den ersten Fehler,
  7: Ein Breakpoint hier, ein Blick dorthin,     1 von 8 Zeilen
  5: der Rest ist meistens nur Erzaehler.
  4: Ich lese nur den ersten Fehler,
  2: die erste Zeile macht mir Sorgen.
  1: Der Compiler meckert schon am Morgen,
6 von 8 Zeilen
```

Was passiert bei `for (size_t k = puffer.size() - 1; k >= 0; --k)`?

Aufgabe 3. *(ca. 30 min · `blatt3/a3_csv`)* Schreiben Sie eine Klasse `CsvLeser`, die eine Textdatei Datensatz für Datensatz liest. Die Felder eines Datensatzes sind durch ein Trennzeichen getrennt (Vorgabe `;`). Der Leser hält dazu einen geöffneten `std::ifstream` als Attribut.

- Der Konstruktor erhält den Dateinamen und optional das Trennzeichen. Ein String soll nicht unbemerkt zu einem `CsvLeser` werden. Lässt sich die Datei nicht öffnen, wird `std::runtime_error` mit dem Text `Datei nicht lesbar:` und dem Dateinamen geworfen.
- `naechste(felder)` liest den nächsten Datensatz in einen `std::vector<std::string>` des Aufrufers und liefert `true`. Leere Zeilen werden übersprungen. Ein Trennzeichen am Zeilenende erzeugt ein leeres letztes Feld. Am Dateiende liefert die Methode `false` und lässt `felder` unverändert.
- `zeile` liefert die Zeilennummer (ab 1, leere Zeilen mitgezählt) des zuletzt gelieferten Datensatzes, `gelesen` die Anzahl gelieferter Datensätze, `trenner` das Trennzeichen.
- Ein `CsvLeser` kann nicht kopiert oder zugewiesen werden. Die Datei wird geschlossen, wenn der Leser zerstört wird.

Testen Sie mit `test_csvleser.cpp`. Der Test legt seine Dateien selbst an. Schreiben Sie ein Programm `notenliste`, das aus einer CSV-Datei (Vorgabe `noten.csv`, Option `-trenner=X` mit genau einem Zeichen) Zeilen der Form `Name;Punkte` liest. Es gibt den Namen linksbündig in Breite 10 und die Punkte rechtsbündig in Breite 4 aus (`std::stoi`). Datensätze mit falscher Feldzahl meldet es mit ihrer Zeilennummer auf `std::cerr`. Am Ende stehen die Anzahl der Datensätze und die Punktsumme.

```
$ ./notenliste
Susi        87
Rudi        45
Zeile 4 uebersprungen
Bodo        60
Datensaetze: 4, Summe: 192
```

Brauchen Sie einen eigenen Destruktor, damit die Datei geschlossen wird? Begründen Sie es mit dem Lebenszyklus der Attribute (Skript 4.1).

Aufgabe 4. *(ca. 15 min, auf Papier · `blatt3/a4_puffer/puffer.cpp`)* Gegeben ist:

```
class Puffer {
public:
    explicit Puffer(size_t n) : _n(n), _p(new int[n]{}) { cout << "K" << _n << " "; }
    ~Puffer() { cout << "D" << _n << " " << flush; delete[] _p; }
    int& at(size_t i) { return _p[i]; }
    size_t _n;
    int* _p;
};

int main() {
    Puffer a(3);                              // 1
    Puffer b(5);                              // 2
    a.at(0) = 7;                              // 3
    Puffer c = a;                             // 4
    c.at(0) = 9;                              // 5
    b = a;                                    // 6
    cout << a.at(0) << " " << b._n << endl;   // 7
}                                             // 8
```

1. Zeichnen Sie das Speicherbild nach Zeile 6: alle drei Objekte und alle Freispeicherbereiche mit Inhalt (wie Skript Abb. 4.1).
2. Was gibt Zeile 7 aus?
3. Wie viele Bytes gehen verloren, und durch welche Zeile?
4. In welcher Reihenfolge laufen die Destruktoren in Zeile 8, und bei welchem stürzt das Programm ab?

5. Ergänzen Sie die Klasse nach der Dreierregel mit Copy & Swap. Ergänzen Sie dabei eine Ausgabe `C` im Kopierkonstruktor. Was gibt das Programm dann aus?

Prüfen Sie Ihre Antworten, indem Sie das Programm ausführen und mit valgrind laufen lassen.